

به نام خدا



Model Layer

Core Entities:

- Product
- Customer
- LoyalCustomer

Shopping Components:

- Cart
- CartItem
- CartStatus

Supporting:

- UnitType

Transaction:

- Invoice
- PaymentMethod



UI Layer

Entry Point UI:

- ConsoleUI (Login / Signup / Exit)

Customer Section:

- CustomerPanel

Admin Section:

- AdminPanel
- ProductManager
- CustomerManager



Service Layer

Core Service:

- Store

Support Services:

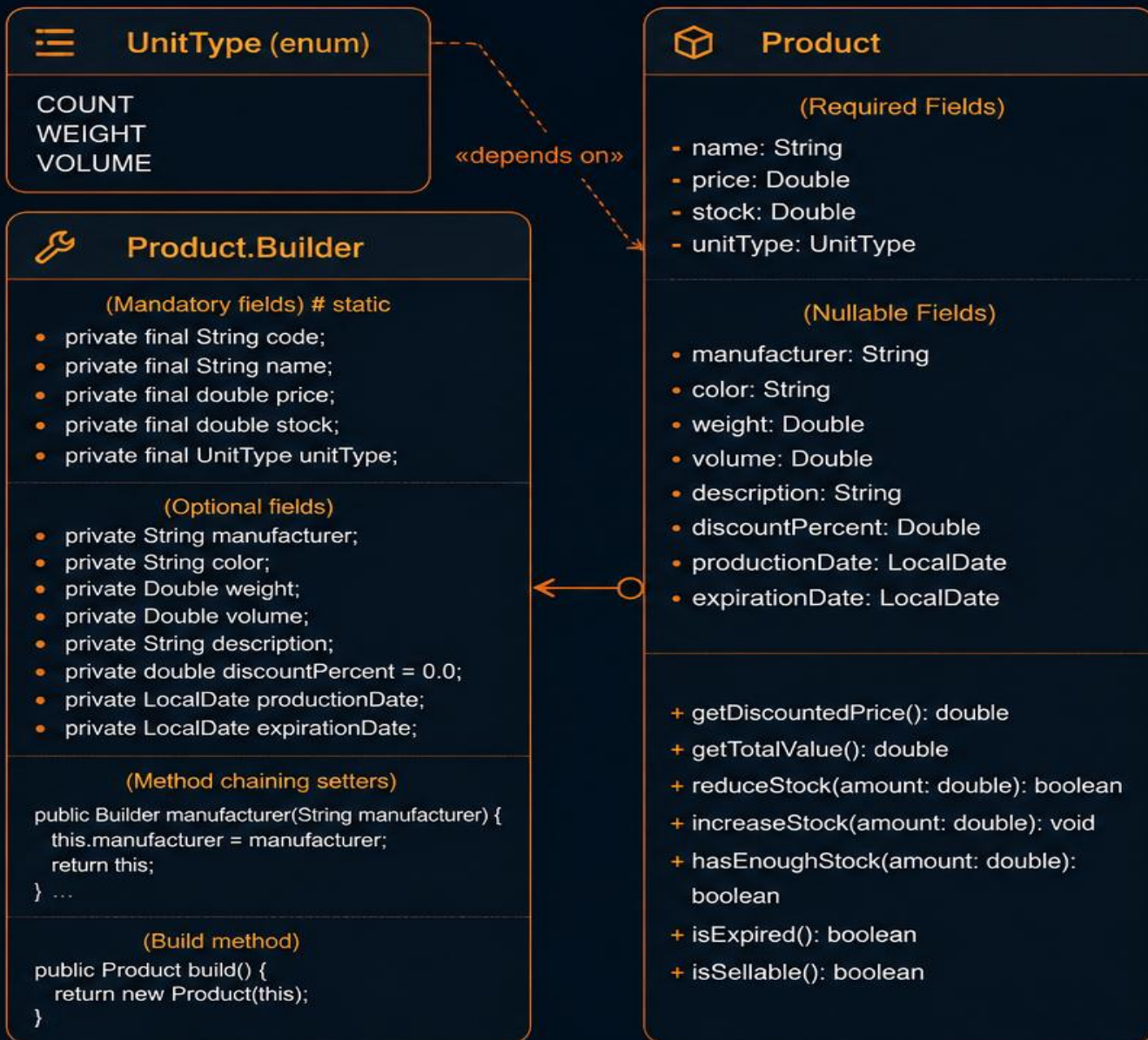
- RandomDataGenerator
- Logger



Utility Layer

- Constants
- InputValidator

Product Class Design



کلاس **Product** یکی از اصلی‌ترین موجودیت‌های سیستم است که اطلاعات مربوط به کالاها را نگهداری می‌کند.



این کلاس شامل مشخصات پایه، موجودی و برخی ویژگی‌های اختیاری محصول می‌باشد.



همچنین عملیات مهمی مانند مدیریت موجودی، بررسی انقضا و محاسبه قیمت پس از تخفیف در آن تعریف شده است.



برای ساخت اشیاء این کلاس نیز از الگوی طراحی **Builder** استفاده شده است.

Customer & LoyalCustomer Design



Customer

- name: String
- phone: String

- + canBuyOnCredit(): boolean → return false
- + canReturnItem(): boolean → return false



LoyalCustomer

- membershipCode: String
- joinDate: LocalDate
- credit: double
- debt: double

- + canBuyOnCredit(): boolean
- + isCreditAvailable(): boolean
- + canReturnItem(): boolean
- + addDebt(): void
- + addCredit(): void
- + payDebt(): void

LoyalCustomer Extends Customer



کلاس **Customer** نمایانگر مشتری عادی سیستم است و اطلاعات پایه‌ای مانند نام و شماره تماس را نگهداری می‌کند.



کلاس **LoyalCustomer** با استفاده از وراثت از **Customer** توسعه یافته و امکاناتی مانند خرید اعتباری، بازگرداندن کالا و مدیریت اعتبار مالی را فراهم می‌کند.



این ساختار باعث استفاده مجدد از کد و توسعه‌پذیری بهتر سیستم شده است.



کلاس **Customer** نمایانگر مشتری عادی سیستم است و اطلاعات پایه‌ای مانند نام و شماره تماس را نگهداری می‌کند. این کلاس رفتارهای عمومی مشتریان را تعریف کرده و به عنوان کلاس پایه برای انواع مختلف مشتریان مورد استفاده قرار می‌گیرد.



کلاس **LoyalCustomer** با استفاده از وراثت از **Customer** توسعه یافته و امکانات ویژه‌ای مانند خرید اعتباری، بازگرداندن کالا و مدیریت اعتبار مالی را فراهم می‌کند.

Shopping Cart Design



این بخش مسئول مدیریت فرآیند خرید در سیستم است. سبد خرید شامل مجموعه‌ای از اقلام خرید، اطلاعات مشتری و وضعیت فعلی سبد می‌باشد. همچنین عملیات افزودن، حذف و نهایی‌سازی خرید در این بخش مدیریت می‌شود.



Cart item

نگهداری محصول انتخاب‌شده
به همراه تعداد آن و
محاسبه قیمت آیتم.



Cart status

مشخص می‌کند سبد
خرید باز است یا فرآیند
خرید آن نهایی شده است.



CartItem

- product : Product
- quantity : double

+ getTotalPrice()
+ setQuantity()



Cart

1 - items: List<CartItem> {FK reference from CartItem}
- customer: Customer {rel: one to one}
- createDate: LocalDateTime
- status: CartStatus

+ addItem(product: Product, quantity: double): void
+ removeItem(productCode: String): void
+ getTotalAmount(): double
+ checkout(paymentMethod: PaymentMethod): Invoice

it is Dependency

it is Dependency



CartStatus (enum)

- OPEN
- CLOSED



PaymentMethod (enum)

- CASH
- CREDIT



Cart

نگهداری اقلام خرید، مدیریت
وضعیت سبد و نهایی‌سازی
فرآیند خرید.



payment method

تعیین نوع پرداخت مورد
استفاده در هنگام ثبت
فاکتور.



Association (1 to many)



Dependency

Invoice Design



Invoice

- final paymentMethod : PaymentMethod
- final id : String
- final items : List<CartItem>
- final customer : Customer
- final dateTime : LocalDateTime
- final finalAmount : double

- + getId() : String
- + getItems() : List<CartItem>
- + getCustomer() : Customer
- + getDateTime() : LocalDateTime
- + getFinalAmount() : double
- + getPaymentMethod() : PaymentMethod
- + toString() : String



کلاس **Invoice** نمایانگر فاکتور نهایی خرید است که پس از ثبت سفارش ایجاد می‌شود. این کلاس اطلاعاتی مانند شناسه فاکتور، مشتری، اقلام خریداری‌شده، روش پرداخت، زمان ثبت و مبلغ نهایی را نگهداری می‌کند.



همچنین با استفاده از یک نسخه محافظت‌شده از اطلاعات سبد خرید، از تغییرات ناخواسته پس از صدور فاکتور جلوگیری می‌شود.



Immutable Design

بیشتر فیلدهای این کلاس به صورت **final** تعریف شده‌اند تا پس از صدور فاکتور، اطلاعات آن قابل تغییر نباشد.



Defensive Copying

در زمان ایجاد فاکتور، یک کپی مستقل از اقلام سبد خرید ذخیره می‌شود تا تغییرات بعدی در سبد خرید روی اطلاعات فاکتور تأثیر نگذارد.



Read-Only Access

لیست اقلام خرید به صورت **unmodifiableList** برگردانده می‌شود تا از تغییر مستقیم داده‌ها جلوگیری شود.



کلاس **Invoice** در واقع خروجی نهایی فرآیند خرید در سیستم است. در این بخش اطلاعات تراکنش به صورت یک رکورد مستقل ذخیره می‌شود تا حتی در صورت تغییرات بعدی مانند برگشت کالا توسط مشتری ویژه، سابقه فاکتور بدون تغییر و قابل استناد باقی بماند.

پایان